Instruction for running IPFS Using Flask

1. Pre-requisites:
   Linux Ubuntu OS: 20.04 or later
   go: 1.18+
   docker and docker-compose: latest version

2. Pull ipfs-kubo docker image:
   docker pull ipfs/kubo

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$     docker pull ipfs/kubo
Using default tag: latest
latest: Pulling from ipfs/kubo
Digest: sha256:41ad6d8c6ecaa3ba163d808a59679fa63cc44b50547179cef391d27a028d005e
Status: Image is up to date for ipfs/kubo:latest
docker.io/ipfs/kubo:latest
```

3. Create a two node folders
   mkdir ipfs_test
   mkdir ipfs_test/node1
   mkdir ipfs_test/node2

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test/ipfs_test$  mkdir ipfs_test
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test/ipfs_test$ mkdir ipfs_test/node1
```

4. Generate swarm.key:
   cd ipfs_test
   git clone https://github.com/Kubuxu/go-ipfs-swarm-key-gen

   ##Next command will Overwrite existing swarm.key
   go run go-ipfs-swarm-key-gen/ipfs-swarm-key-gen/main.go > swarm.key

5. Run node1 and node2 by executing following scripts:
   ./node1/startnode1.sh
   ./node2/startnode2.sh

   You can check running nodes:
   docker ps

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ docker ps
CONTAINER ID   IMAGE              COMMAND                  CREATED        STATUS                  PORTS

                                          NAMES
e6522ed495d2   ipfs/kubo:latest   "/sbin/tini -- /usr/…"   35 hours ago   Up 35 hours (healthy)   8081/
tcp, 0.0.0.0:4002->4001/tcp, 0.0.0.0:4002->4001/udp, :::4002->4001/tcp, :::4002->4001/udp, 127.0.0.1:50
02->5001/tcp, 127.0.0.1:8082->8080/tcp   ipfs_node2
57e8564e2d48   ipfs/kubo:latest   "/sbin/tini -- /usr/…"   35 hours ago   Up 35 hours (healthy)   0.0.0
.0:4001->4001/tcp, 0.0.0.0:4001->4001/udp, :::4001->4001/tcp, :::4001->4001/udp, 127.0.0.1:5001->5001/t
cp, 8081/tcp, 127.0.0.1:8081->8080/tcp   ipfs_node1
```

6. Set Bootstrap Peer

Get nodes id:

docker exec ipfs_node1 ipfs id

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ docker exec ipfs_node1 ipfs id
{
        "ID": "12D3KooWBoa7rYY3RTEB3zFsp6WBS57DCY4grG6VBPxAU5HgPrKz",
        "PublicKey": "CAESIB2DTh7RcCnSK6hMEZyYNy8+DdObFBjTYafFOB/ZCPqR",
        "Addresses": [
                /ip4/127.0.0.1/tcp/4001/p2p/12D3KooWBoa7rYY3RTEB3zFsp6WBS57DCY4grG6VBPxAU5HgPrKz",
                "/ip4/127.0.0.1/udp/4001/quic-v1/p2p/12D3KooWBoa7rYY3RTEB3zFsp6WBS57DCY4grG6VBPxAU5HgPr
Kz",
                "/ip4/127.0.0.1/udp/4001/quic-v1/webtransport/certhash/uEiCKhZfjSNjqwAln0XgkLW2o_68gRll
8RurUf6_P0V_KAQ/certhash/uEiA8L1uHs20BKK9PpAA7s5AF58q8hkBQTw18079KXcCR_w/p2p/12D3KooWBoa7rYY3RTEB3zFsp6
WBS57DCY4grG6VBPxAU5HgPrKz",
                "/ip4/127.0.0.1/udp/4001/webrtc-direct/certhash/uEiCpi04OcJFYnJsmKMn1uSgHkif__m4RlBx6XV
rEVZId0g/p2p/12D3KooWBoa7rYY3RTEB3zFsp6WBS57DCY4grG6VBPxAU5HgPrKz",
                "/ip4/172.17.0.2/tcp/4001/p2p/12D3KooWBoa7rYY3RTEB3zFsp6WBS57DCY4grG6VBPxAU5HgPrKz",
                "/ip4/172.17.0.2/udp/4001/quic-v1/p2p/12D3KooWBoa7rYY3RTEB3zFsp6WBS57DCY4grG6VBPxAU5HgP
rKz",
```

docker exec ipfs_node2 ipfs id

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ docker exec ipfs_node2 ipfs id
{
        "ID": "12D3KooWE8BPueXBzH72bwGiqBtaaDM5rveKVtRdBxgDHMvCMah3",
        "PublicKey": "CAESID//BWrSzcVpAFWEeZyMAqZMuH1gN1CZn86BL5mtCGKm",
        "Addresses": [
                /ip4/127.0.0.1/tcp/4001/p2p/12D3KooWE8BPueXBzH72bwGiqBtaaDM5rveKVtRdBxgDHMvCMah3",
                "/ip4/127.0.0.1/udp/4001/quic-v1/p2p/12D3KooWE8BPueXBzH72bwGiqBtaaDM5rveKVtRdBxgDHMvCMa
h3",
                "/ip4/127.0.0.1/udp/4001/quic-v1/webtransport/certhash/uEiCTXmi4pkbXGLBzCta4Ea7hyOE32hK
aVk-X9N1nl0EBEg/certhash/uEiCudJLdMGcoGhXl3PLhVQaZzeJnM-07NShj1OPrDaDTxw/p2p/12D3KooWE8BPueXBzH72bwGiqB
taaDM5rveKVtRdBxgDHMvCMah3",
                "/ip4/127.0.0.1/udp/4001/webrtc-direct/certhash/uEiA0JuhEadpLczNfyyh7kgE5ecqf8tBFiRtXZh
uCor31aQ/p2p/12D3KooWE8BPueXBzH72bwGiqBtaaDM5rveKVtRdBxgDHMvCMah3",
                "/ip4/172.17.0.3/tcp/4001/p2p/12D3KooWE8BPueXBzH72bwGiqBtaaDM5rveKVtRdBxgDHMvCMah3",
                "/ip4/172.17.0.3/udp/4001/quic-v1/p2p/12D3KooWE8BPueXBzH72bwGiqBtaaDM5rveKVtRdBxgDHMvCM
```

Remove all default bootstrap nodes:

docker exec ipfs_node1 ipfs bootstrap rm all

Add bootstrap node (replace with node address):

docker exec ipfs_node1 ipfs bootstrap add @node2_address

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ docker exec ipfs_node1 ipfs bootstra
p add /ip4/127.0.0.1/tcp/4001/p2p/12D3KooWE8BPueXBzH72bwGiqBtaaDM5rveKVtRdBxgDHMvCMah3
added /ip4/127.0.0.1/tcp/4001/p2p/12D3KooWE8BPueXBzH72bwGiqBtaaDM5rveKVtRdBxgDHMvCMah3
```

docker exec ipfs_node2 ipfs bootstrap add @node1_address

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test/ipfs_test$ docker exec ipfs_node2 ipf
s bootstrap add /ip4/127.0.0.1/tcp/4001/p2p/12D3KooWBoa7rYY3RTEB3zFsp6WBS57DCY4grG6VBPxAU5HgPrKz
added /ip4/127.0.0.1/tcp/4001/p2p/12D3KooWBoa7rYY3RTEB3zFsp6WBS57DCY4grG6VBPxAU5HgPrKz
```

7. Nodes are ready and you can test upload and get operation if you want.

Add file:
a) You can save some files under the docker folder. for example ./node1/data. As docker can only access attached local folders.

docker exec ipfs_node1 ipfs add /data/ipfs/testfile.txt

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ docker exec ipfs_node1 ipfs add /dat
a/ipfs/testfile.text
added QmcC4fp443QNcD9dMq47MYHW5resA4zXP2iPpN5BKKS5m3 testfile.text
```

b) You can use curl to input files in a local folder rather than a docker folder. For example "test_file.txt" under the current folder.
curl -F file=@test_file.txt -X POST "http://127.0.0.1:5001/api/v0/add"

Get file:
a) You can read content of file by using 'ipfs cat' given a CID

docker exec ipfs_node2 ipfs cat
QmcC4fp443QNcD9dMq47MYHW5resA4zXP2iPpN5BKKS5m3

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ docker exec ipfs_node2 ipfs cat QmcC
4fp443QNcD9dMq47MYHW5resA4zXP2iPpN5BKKS5m3
Hi, Manoj
```

or you can download the file by using 'ipfs get' then save it to the docker data folder.

docker exec ipfs_node2 ipfs get
QmcC4fp443QNcD9dMq47MYHW5resA4zXP2iPpN5BKKS5m3 -o
/data/ipfs/outputfile.txt

b) You can use curl to query file from ipfs

curl -X POST
"http://127.0.0.1:5001/api/v0/cat?arg=QmcC4fp443QNcD9dMq47MYHW5resA4zXP2iPp
N5BKKS5m3"

or save data to local file

curl -X POST
"http://127.0.0.1:5001/api/v0/cat?arg=QmcC4fp443QNcD9dMq47MYHW5resA4zXP2iPp
N5BKKS5m3" > myoutputfile.txt

8. Start the Flask Server
   Pre-requisites: install flask

   python3 ipfs_server.py

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ python3 ipfs_server.py
 * Serving Flask app 'ipfs_server'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI
server instead.
 * Running on all addresses (0.0.0.0)
 * Running on http://127.0.0.1:8000
 * Running on http://192.168.15.2:8000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 616-608-866
127.0.0.1 - - [16/Feb/2026 00:55:34] "POST /ipfs/add HTTP/1.1" 200 -
127.0.0.1 - - [16/Feb/2026 00:55:34] "GET /ipfs/cat/QmbGi4y5wecRJiAqNqDYF6v1bqa51xNPYcyN1GbemYuytJ HTTP
/1.1" 200 -
```

9. Run Client
   python3 client.py

uploaded test_file.txt

```
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ ls
client.py           go-ipfs-swarm-key-gen  ipfs_server.py  node1   __pycache__   swarm.key
docker-compose.yml  ipfs                   ipfs_test       node2   RPC_ipfs.py   test_file.txt
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ cat test_file.txt
hi there, Manoj is testing ipfs.
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ python3 client.py
Uploaded CID: QmVkZs9bfvwUQbean1We3GzQcjqcmDTqa6puU2XKfN274E
File downloaded successfully
instructor@instructor-virtual-machine:~/Documents/Manoj/ipfs_test$ cat downloaded_test_file.txt
hi there, Manoj is testing ipfs.
```

Downloaded file

Now, the file is uploaded and also downloaded through nodes.

10. Stop nodes:
    docker stop ipfs_node1
    docker stop ipfs_node2